BEGINNER BUILD GUIDE

NetWatchM

Real-Time Network Threat Monitor

A complete, step-by-step guide for beginners —
from zero to a fully running network security monitor.

What you will build: A live network monitor that watches all traffic on your network and alerts you when something suspicious is detected.

Version: 0.1.0 · **Platform:** Linux (Ubuntu / Debian) · **Date:** February 2026

Table of Contents

Phase 1	What is NetWatchM and How Does It Work?	Overview & architecture
Phase 2	Before You Begin — Prerequisites	Checklist & installs
Phase 3	Getting the Code	Clone & explore
Phase 4	Installing NetWatchM	Automated installer walkthrough
Phase 5	Understanding the Configuration File	Every setting explained
Phase 6	Running NetWatchM for the First Time	Interactive & service modes
Phase 7	Reading the Dashboard	Keyboard shortcuts & panels
Phase 8	Understanding Alerts	What each alert means
Phase 9	The Web Dashboard (HTML Report)	Browser-based device viewer
Phase 10	Running as a Background Service	Systemd setup & management
Phase 11	Querying the Device Inventory	CLI & CSV export
Phase 12	Setting Up Email Alerts	Gmail App Password setup
Phase 13	Running the Tests	Verify everything works
Phase 14	Troubleshooting Common Problems	Step-by-step fixes
Phase 15	Log Management & Disk Protection	Preventing log saturation
Phase 16	Service-Down Email Alerts	Get notified when a service stops
Phase 17	PowerShell Log Watcher	Real-time HIGH alert detection
Appendix	Glossary & Quick Reference	Key terms & commands

PHASE 1

What is NetWatchM and How Does It Work?

Understand what the tool does before you build it. No commands yet — just concepts.

What Does NetWatchM Do?

NetWatchM is a **network security monitor** that runs quietly in the background on your Linux computer. It watches every packet of data that travels through your network interface and raises an **alert** when it spots something dangerous.

Think of it like a smoke detector for your network — it runs silently until it notices something wrong, then it alerts you immediately.

It can detect four types of threats:

Threat Type	What it means	Example
Port Scan	Someone probing many ports rapidly	A hacker mapping your network with nmap
Brute Force	Repeated login attempts	A bot hammering your SSH password
Exfiltration	Huge data upload in a short time	A compromised PC uploading files to the internet
New Device	Unknown device joins the network	A stranger's phone connecting to your WiFi

How Does It Work? (The Big Picture)

NetWatchM uses **tshark** (the command-line version of Wireshark) to capture packets, then analyses them in real time using Python. Here is the flow from packet to alert:

```
Your Network Interface (e.g. enp6s0) │ ▼ [ tshark subprocess ] ← captures raw packets, outputs one JSON line per packet │ ▼ [ capture.py ] ← reads each JSON line, creates a Packet object │ ▼ [ 4 Detectors ] ← each packet is checked by all four detectors ├── PortScanDetector ├── BruteForceDetector ├── ExfiltrationDetector └── NewIPDetector │ (if suspicious → Alert) ▼ [ Alert Handlers ] ← delivers the alert via all enabled channels ├── Terminal (coloured text on screen) ├── Log file (/var/log/netwatchm/) ├── Sound (beep!) └── Email (Gmail) │ ▼ [ Device Inventory ] ← every device seen is recorded in inventory.json │ ▼ [ report.html ] ← browser dashboard — refreshes every 30 seconds
```

Key Technologies Used

Technology	What it is	Why NetWatchM uses it
Python 3.12	Programming language	Everything is written in Python
asyncio	Python's async library	Handles many tasks at once without threads
tshark	CLI packet capture tool	The engine that reads raw network traffic
Rich	Python terminal library	The colourful dashboard you see on screen
uv	Python package manager	Installs dependencies and runs the app
systemd	Linux service manager	Keeps NetWatchM running after reboot

Phase 1 Complete — You now know:

- ✔ What NetWatchM does (real-time network threat detection)
- ✔ The four threats it detects (port scan, brute force, exfiltration, new device)
- ✔ How data flows from network → packet → alert → dashboard

PHASE 2

Before You Begin — Prerequisites

2

Check and install everything NetWatchM needs before you touch the code.

What You Need

- [ ] A Linux computer running Ubuntu 22.04 / 24.04 or any Debian-based distro
- [ ] A terminal (press `Ctrl` + `Alt` + `T` to open one)
- [ ] sudo (administrator) access
- [ ] An internet connection for the initial install
- [ ] Python 3.12 or newer
- [ ] tshark (Wireshark's command-line tool)
- [ ] uv (a fast Python package manager)

Step-by-Step: Check and Install Prerequisites

1 Check your Python version

Open a terminal and type:

```
python3 --version
```

You should see something like `Python 3.12.x`. If the number is lower than 3.12, you need to upgrade Python before continuing.

> ¡ **How to upgrade Python on Ubuntu**
>
> `sudo apt update && sudo apt install python3.12 python3.12-dev`

2 Install tshark

tshark is the packet capture engine that NetWatchM uses to read network traffic. Install it with:

```
sudo apt update
sudo apt install tshark
```

During the install, a dialog will ask:

"Should non-superusers be able to capture packets?"

> ⚠ **Choose "Yes" only if you plan to run without sudo**
>
> For simplicity, choose **No** — you will run NetWatchM with sudo anyway. You can always change this later.

Verify it installed correctly:

```
tshark --version
```

You should see `TShark (Wireshark) x.x.x`.

3 Install uv (Python package manager)

uv is a fast modern Python package manager. It replaces pip and venv in one tool.

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

This downloads and installs uv into `~/.local/bin/`. Now tell your shell where to find it:

```
export PATH="$HOME/.local/bin:$PATH"
```

To make this permanent (so it works every time you open a terminal), add it to your shell profile:

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

Verify uv is working:

```
uv --version
```

4 Find your network interface name

You need to know the name of your network card so NetWatchM knows what to monitor. Run:

```
ip addr
```

Look for the interface with your local IP address (usually starts with `192.168.` or `10.`). Common names are:

Name	What it usually is
`enp6s0`, `eth0`	Wired Ethernet
`wlan0`, `wlp2s0`	Wireless / WiFi
`lo`	Loopback (localhost only — not useful for monitoring)

Write down your interface name — you will use it in the config file.

Phase 2 Complete — You now have:

- ✓ Python 3.12+ confirmed
- ✓ tshark installed and working
- ✓ uv installed and on your PATH
- ✓ Your network interface name noted

PHASE 3

Getting the Code

3

Download the NetWatchM source code and understand how the project is laid out.

1 Go to your projects folder

```
cd ~
mkdir -p ai-projects
cd ai-projects
```

This creates a folder called `ai-projects` in your home directory and moves you into it.

2 Clone the repository

```
git clone https://github.com/yourrepo/netwatchm.git
```

This downloads all the source code into a folder called `netwatchm/`.

¡ **No git installed?**

`sudo apt install git`

3 Explore the project layout

Move into the project folder and look around:

```
cd netwatchm
ls -la
```

You will see these important files and folders:

File / Folder	What it is
`src/netwatchm/`	All the Python source code
`tests/`	57 automated tests
`netwatchm.yaml.example`	Template configuration file
`install.sh`	One-command Linux installer
`pyproject.toml`	Python project metadata and dependencies
`assets/alert.wav`	The beep sound played for HIGH alerts
`report.html`	Browser-based inventory dashboard

Phase 3 Complete — You now have:

- ✓ The NetWatchM source code on your machine
- ✓ An understanding of what each file and folder does

PHASE 4

Installing NetWatchM

Use the automated installer to set up everything in one command.

4

What the Installer Does

The `install.sh` script automates the entire setup. Before you run it, here is exactly what it will do — no surprises:

#	Step	Result
1	Verifies Python ≥ 3.12	Stops with an error if too old
2	Verifies tshark is installed	Tries to install it automatically if missing
3	Installs uv if missing	Downloads to `/root/.local/bin/uv`
4	Runs `uv sync`	Installs all Python dependencies into `.venv/`
5	Creates `/etc/netwatchm/netwatchm.yaml`	Copies the example config (only if file doesn't exist)
6	Creates log and data directories	`/var/log/netwatchm/` and `/var/lib/netwatchm/`
7	Prompts for Gmail App Password	Saves it securely to `/etc/netwatchm/env`
8	Installs systemd service	NetWatchM starts automatically on every boot

1 Run the installer

From inside the `netwatchm/` folder:

```
sudo bash install.sh
```

Enter your password when prompted. Then watch the output — each step is shown with a green `[INFO]` label.

2 What success looks like

A successful install produces output like this:

```
[INFO] Python 3.12 OK
[INFO] tshark found: /usr/bin/tshark
[INFO] Installing uv...
[INFO] uv found: /root/.local/bin/uv
[INFO] Installing Python dependencies...
Resolved 15 packages in 0.71ms
[INFO] Creating /etc/netwatchm/netwatchm.yaml...
[INFO] Installing systemd service...
Wrote /etc/systemd/system/netwatchm.service
[INFO] NetWatchM installed successfully!
```

3 Verify the service is running

```
systemctl status netwatchm
```

Look for `Active: active (running)` in the output. If you see that, NetWatchM is already monitoring your network!

> 💡 **View live logs at any time**
>
> `journalctl -u netwatchm -f`
>
> Press `Ctrl` + `C` to stop following the log.

4 What got created on your system

Path	What it is
`/etc/netwatchm/netwatchm.yaml`	Main configuration file
`/etc/netwatchm/env`	Secret environment variables (email password)
`/var/log/netwatchm/netwatchm.log`	Alert log file
`/var/lib/netwatchm/inventory.json`	Device inventory database
`/etc/systemd/system/netwatchm.service`	Systemd service definition

Phase 4 Complete — You now have:

- ✓ NetWatchM installed and running as a system service
- ✓ Config file at `/etc/netwatchm/netwatchm.yaml`
- ✓ Inventory database at `/var/lib/netwatchm/inventory.json`
- ✓ Logs flowing to `/var/log/netwatchm/netwatchm.log`

PHASE 5

Understanding the Configuration File

5

Learn every setting and what changing it means for your monitoring.

The configuration file lives at `/etc/netwatchm/netwatchm.yaml`. Open it with any text editor:

```
sudo nano /etc/netwatchm/netwatchm.yaml
```

Section 1 — Basic Settings

```
interface: auto          # Which network card to watch
baseline_period: 300     # Learning period in seconds (5 minutes)
```

Setting	Default	What it does
`interface`	`auto`	Set to `auto` to let NetWatchM pick the right NIC automatically, or set an exact name like `enp6s0` or `wlan0`.
`baseline_period`	`300`	When NetWatchM first starts, it spends this many seconds silently learning which devices are normally on the network. After this period, any new device triggers an alert. Lower this number for faster alerting; raise it if your network is large.

Section 2 — Threat Detection Thresholds

```
thresholds:
  port_scan:
    ports_per_window: 15     # alert after hitting this many
distinct ports
    window_seconds: 10       # within this time window

  brute_force:
    attempts_per_window: 10
# alert after this many connection attempts
    window_seconds: 30       # within this time window
    ports: [22, 3389, 21, 3306, 5900]  # ports to watch (SSH,
RDP, FTP, MySQL, VNC)

  exfiltration:
    bytes_per_window: 10485760   # 10 MB — alert if one device
sends this much
    window_seconds: 60           # within 60 seconds

  new_ip:
    enabled: true            # alert when a device never seen
before appears
```

💡 **Tuning tip for home networks**

If you get too many false alerts (e.g. your own computer triggering port-scan alerts during a software update), raise `ports_per_window` to 25–30. If you want earlier warnings, lower it to 10.

Section 3 — Alert Channels

```
alerts:
  terminal: true           # show alerts on screen (disable
with --no-ui)

  log:
    enabled: true
    path: /var/log/netwatchm/netwatchm.log
    max_bytes: 10485760    # rotate log at 10 MB
    backup_count: 5        # keep 5 old log files

  sound:
    enabled: true
```

```
    file: assets/alert.wav
    min_level: HIGH        # only beep for HIGH or CRITICAL

  email:
    enabled: false         # set to true to enable email alerts
    smtp_host: smtp.gmail.com
    smtp_port: 587
    username: you@gmail.com
    recipient: you@gmail.com
    min_level: HIGH
    cooldown_seconds: 300  # wait 5 min before sending same
alert type again
```

Section 4 — Device Inventory

```
inventory:
  enabled: true
  persist_interval: 60    # save inventory.json every 60
seconds
  dns_timeout: 2          # seconds to wait for reverse DNS
lookup
  dns_cache_ttl: 300      # cache failed DNS for 5 minutes
  export_dir: .           # where to save CSV exports (. =
current folder)
```

⚠ **Always restart after changing the config**

`sudo systemctl restart netwatchm`

Changes to `netwatchm.yaml` do not take effect until the service restarts.

Phase 5 Complete — You now understand:

- ✓ How to set your network interface manually
- ✓ What each detection threshold controls
- ✓ How to enable/disable each alert channel
- ✓ How the device inventory is saved

PHASE 6

Running NetWatchM for the First Time

Start the monitor manually in interactive mode so you can see exactly what it does.

¡ **Two ways to run NetWatchM**

Interactive mode — you run it in a terminal and see the live dashboard.
Service mode — systemd runs it in the background, no dashboard.
For your first run, use interactive mode so you can see what happens.

1 Stop the background service first

The service might already be running. Stop it temporarily so you can run interactively without conflicts:

```
sudo systemctl stop netwatchm
```

2 Run NetWatchM interactively

```
cd /home/$USER/ai-projects/netwatchm
sudo uv run netwatchm --config /etc/netwatchm/
netwatchm.yaml
```

The Rich dashboard will appear immediately. You will see your network interface name, a threat level indicator, and an alert panel.

3 Override the interface from the command line

If you want to specify the interface without editing the config file:

```
sudo uv run netwatchm --config /etc/netwatchm/
netwatchm.yaml --interface enp6s0
```

4 Quit when you are done

Press `Q` to quit cleanly. NetWatchM will save the inventory to disk before exiting.

5 Restart the background service

```
sudo systemctl start netwatchm
```

All Command-Line Options

Option	Example	What it does
`--config PATH`	`--config /etc/netwatchm/netwatchm.yaml`	Which config file to use
`--interface NAME`	`--interface wlan0`	Override the network interface
`--no-ui`	`--no-ui`	Disable the dashboard (log only)
`--log-level LEVEL`	`--log-level DEBUG`	Set verbosity: DEBUG / INFO / WARNING / ERROR
`--install-service`	`--install-service`	Install/update the systemd service and exit

Phase 6 Complete — You can now:

- ✔ Run NetWatchM interactively with the live dashboard
- ✔ Override the interface from the command line

- ✔ Quit cleanly with the Q key

PHASE 7

Reading the Dashboard

7

Understand every panel and keyboard shortcut in the terminal UI.

Dashboard Layout

```
┌──────────────────────────────────────────────────────────┐
│ NetWatchM │ Interface: enp6s0 │ Threat: LOW │ Pkts/s: 142 │
├──────────────────────────────────────────────────────────┤
│ RECENT ALERTS │ │ [16:45:12] MEDIUM NEW_IP 192.168.1.250 — new
device detected │ │ [16:31:05] HIGH PORT_SCAN 203.0.113.42 — 18
ports in 10s │ │ │
├──────────────────────────────────────────────────────────┤
│ Traffic: ↑ 12.1 MB ↓ 1.3 MB │ Press I=Inventory Q=Quit │
└──────────────────────────────────────────────────────────┘
```

Panel Descriptions

Panel Element	What it shows
Interface	The network card currently being monitored
Threat level	The current highest active threat: LOW (green) / MEDIUM (yellow) / HIGH (red) / CRITICAL (bold red)
Pkts/s	How many network packets per second are flowing through your interface
Recent Alerts	Scrolling list of the most recent detections with timestamp, level, type, source IP, and description
Traffic	Total bytes sent ↑ and received ↓ since NetWatchM started

Keyboard Shortcuts

Key	Action
`Q`	Quit NetWatchM (saves inventory first)
`I`	Switch to Inventory view — shows all devices seen on the network
`M`	Switch back to main dashboard from Inventory view
`E`	(Inventory view only) Export device list to a CSV file
`/`	(Inventory view only) Start typing to filter devices by IP or hostname
`Esc`	(Inventory view only) Clear the current filter
`Backspace`	(Inventory view only) Delete the last character of the filter

Inventory View Columns

Column	What it shows
IP	The device's IP address
Hostname	The device's name from DNS (may take a few seconds to appear)
MAC	Hardware address (only visible for devices on the same LAN segment)
Threat	The highest threat level this device has triggered
Last Seen	When this device last sent or received a packet
Bytes	Total data sent and received by this device

Phase 7 Complete — You can now:

- ✔ Read every panel in the dashboard

- ✔ Interpret threat levels and alert descriptions
- ✔ Navigate between dashboard and inventory views

PHASE 8

Understanding Alerts

What each type of alert means and how the detection works.

Alert Levels

Level	Colour	What it means
LOW	Green	Normal — no active threats
MEDIUM	Yellow	Worth checking — possibly suspicious
HIGH	Red	Likely malicious — act now
CRITICAL	Bold Red	Confirmed serious threat

PORT_SCAN — HIGH

What triggered it: A single IP address contacted 15+ different ports on your machine within 10 seconds.

What it usually means: Someone is running a port scanner (like nmap) to map which services are running on your network. This is often the first step before an attack.

Alert example:

```
PORT_SCAN from 203.0.113.42: 18 distinct ports in 10s window
```

What to do: Check if the source IP is on your LAN. If it's external, consider blocking it with your firewall (`sudo ufw deny from 203.0.113.42`).

BRUTE_FORCE — HIGH

What triggered it: A single IP made 10+ connection attempts to a sensitive port (SSH=22, RDP=3389, FTP=21, MySQL=3306, VNC=5900) within 30 seconds.

What it usually means: An automated bot is trying to guess your password. This happens constantly on internet-facing servers.

Alert example:

```
BRUTE_FORCE from 198.51.100.7 on port 22: 14 attempts in 30s
window
```

What to do: Install `fail2ban` to automatically block IPs after repeated failures. Use SSH key authentication instead of passwords.

EXFILTRATION — HIGH

What triggered it: A device sent more than 10 MB of data within 60 seconds.

What it usually means: Either a compromised device is uploading data (e.g. a keylogger exfiltrating screenshots), or a legitimate backup/upload running at an unusual time.

Alert example:

```
EXFILTRATION from 192.168.1.45: 15.2 MB sent in 60s window
```

What to do: Identify the device (check the inventory view), then investigate what process is sending that data.

NEW_IP — MEDIUM

What triggered it: An IP address appeared that was never seen during the 5-minute baseline learning period.

What it usually means: A new device joined the network. Could be innocent (a guest's phone) or suspicious (an attacker who gained network access).

Alert example:

```
NEW_IP detected: 192.168.1.250 — not seen during baseline
period
```

What to do: Check the inventory view to see if you can identify the device by its hostname or MAC address.

Phase 8 Complete — You now understand:

- ✓ The four threat levels and what each colour means
- ✓ What triggers each type of alert
- ✓ What each alert type means in practice
- ✓ What action to take for each alert type

PHASE 9

The Web Dashboard (HTML Report)

View your network inventory in a browser with live auto-refresh — running as a permanent service.

The `report.html` file is a self-contained web dashboard that reads `inventory.json` and displays all devices in a searchable, sortable table. It auto-refreshes every 30 seconds — no page reload needed.

> ⚠ **Don't use a terminal to run the web server**
>
> Running `python3 -m http.server` in a terminal means the dashboard **stops the moment you close that terminal**. Use the `netwatchm-web` systemd service instead — it stays up permanently and restarts automatically on reboot or crash.

First-Time Setup

1 **Copy the dashboard file into the data directory**

```
sudo cp /home/$USER/ai-projects/netwatchm/report.html /
var/lib/netwatchm/
```

Both `report.html` and `inventory.json` now live together in `/var/lib/netwatchm/` .

2 Install the web dashboard systemd service

```
sudo cp /home/$USER/ai-projects/netwatchm/netwatchm-
web.service \
        /etc/systemd/system/netwatchm-web.service
sudo systemctl daemon-reload
sudo systemctl enable --now netwatchm-web
```

`enable --now` does two things at once: marks the service to start on every future boot, and starts it immediately right now.

3 Verify it is running

```
systemctl status netwatchm-web
```

You should see `Active: active (running)`. The service definition also declares `Restart=always`, so if it ever crashes systemd will bring it back within 5 seconds — no manual intervention needed.

4 Open the dashboard in your browser

```
http://localhost:8765/report.html
```

The dashboard is now always available at this address — even after a reboot.

> ¡ **The installer handles this automatically**
>
> If you ran `sudo bash install.sh`, steps 1–3 were already done for you. The installer copies `report.html` and installs `netwatchm-web.service` as part of step 9.

Managing the Web Service

Command	What it does
`systemctl status netwatchm-web`	Is it running?
	Start it now

Command	What it does
`sudo systemctl start netwatchm-web`	
`sudo systemctl stop netwatchm-web`	Stop it
`sudo systemctl restart netwatchm-web`	Restart (e.g. after copying a new report.html)
`sudo systemctl enable netwatchm-web`	Start automatically on boot
`sudo systemctl disable netwatchm-web`	Stop starting on boot
`journalctl -u netwatchm-web -f`	Follow live web server logs

Dashboard Features

Feature	How to use it
Search box	Type any IP, hostname, MAC, or port number to filter instantly
Threat filter buttons	Click HIGH / MEDIUM / LOW to show only devices at that risk level
Column sorting	Click any column header to sort; click again to reverse
Port badges	Known ports (SSH, HTTP, DNS…) appear in blue with their service name
+N more	Click to expand a device's full port list
Export CSV button	Downloads the current filtered view as a spreadsheet
Dark / Light toggle	Button in the top-right corner

Feature	How to use it
Auto-refresh ring	The spinning ring completes one full turn every 30 s refresh cycle

Phase 9 Complete — You now have:

- ✓ Web dashboard running as a permanent systemd service (`netwatchm-web`)
- ✓ Dashboard survives reboots and restarts automatically on crash
- ✓ Always accessible at `http://localhost:8765/report.html`
- ✓ Search, filter, sort, and export devices from the browser

PHASE 10

Running as a Background Service

Let systemd manage NetWatchM so it starts automatically and restarts on crashes.

What systemd Does for You

The installer already created the systemd service. systemd gives you:

- [ ] NetWatchM starts automatically when your computer boots
- [ ] If NetWatchM crashes, systemd restarts it automatically
- [ ] All log output is captured and searchable with `journalctl`
- [ ] You can start, stop, restart, or check status with simple commands

Service Management Commands

Command	What it does
`sudo systemctl start netwatchm`	Start the service right now
`sudo systemctl stop netwatchm`	Stop the service
`sudo systemctl restart netwatchm`	Stop then start (use after config changes)
`sudo systemctl enable netwatchm`	Start automatically on every boot
`sudo systemctl disable netwatchm`	Stop starting on boot
`systemctl status netwatchm`	Is it running? When did it start?
`journalctl -u netwatchm -f`	Follow live log output
`journalctl -u netwatchm -n 50`	Show last 50 log lines

Command	What it does
`journalctl -u netwatchm --since "1 hour ago"`	Show logs from the last hour

Applying Configuration Changes

1 Edit the config file

```
sudo nano /etc/netwatchm/netwatchm.yaml
```

2 Save and restart the service

```
sudo systemctl restart netwatchm
```

3 Verify it started successfully

```
systemctl status netwatchm
```

You should see `Active: active (running)`.

Phase 10 Complete — You can now:

- ✓ Start, stop, and restart NetWatchM as a service
- ✓ View live and historical logs with journalctl
- ✓ Apply config changes correctly

PHASE 11

Querying the Device Inventory

Use the CLI to search, sort, and export device data without opening a browser.

NetWatchM has an `inventory` subcommand that reads the saved `inventory.json` and displays it in the terminal. This works even when the service is running — it reads the file directly.

Basic Usage

```
# Show all devices in a Rich table
uv run netwatchm inventory

# Filter by IP or hostname substring
uv run netwatchm inventory --filter 192.168

# Show only devices with "google" in their hostname
uv run netwatchm inventory --filter google

# Sort by threat level (highest first)
uv run netwatchm inventory --sort-by threat

# Sort by total bytes (biggest traffic first)
uv run netwatchm inventory --sort-by bytes

# Sort by when each device was last seen
uv run netwatchm inventory --sort-by last_seen
```

Exporting to CSV

```
# Export to a specific file
uv run netwatchm inventory --export /tmp/my-network.csv

# Export to the terminal (pipe to other tools)
uv run netwatchm inventory --export -

# Filter then export (only HIGH threat devices)
```

```
uv run netwatchm inventory --filter HIGH --export high-
threat.csv
```

Inspecting the Raw JSON

```
# Pretty-print the entire inventory.json
cat /var/lib/netwatchm/inventory.json | python3 -m json.tool

# Count how many devices are tracked
python3 -c "import json; d=json.load(open('/var/lib/netwatchm/
inventory.json')); print(len(d), 'devices')"
```

Phase 11 Complete — You can now:

- ✓ Query the device inventory from the terminal
- ✓ Filter and sort devices by any field
- ✓ Export inventory data to CSV for use in spreadsheets

PHASE 12

Setting Up Email Alerts

12

Get notified by email when a HIGH or CRITICAL threat is detected.

🔒 **Never put your password in the YAML file**

The config file is readable by anyone who can read `/etc/netwatchm/`. Always use the environment variable method described below.

Step 1 — Create a Gmail App Password

1 **Enable 2-Step Verification on your Google account**

Go to **myaccount.google.com → Security → 2-Step Verification** and turn it on. App Passwords only work when 2FA is enabled.

2 **Generate an App Password**

Go to **myaccount.google.com → Security → App passwords**. Choose Mail and Other device. Give it the name "NetWatchM". Google will show you a 16-character password like `abcd efgh ijkl mnop`.

Copy it now — Google only shows it once.

Step 2 — Save the App Password Securely

```
# Save the password to the secure env file
echo "NETWATCHM_EMAIL_PASSWORD=abcdefghijklmnop" | sudo tee /etc/netwatchm/env

# Lock it down so only root can read it
sudo chmod 600 /etc/netwatchm/env
```

Step 3 — Enable Email in the Config

Edit `/etc/netwatchm/netwatchm.yaml` and change the email section:

```
email:
    enabled: true                    # was false — change to
true
    smtp_host: smtp.gmail.com
    smtp_port: 587
    username: yourname@gmail.com     # your Gmail address
    password: ""                     # leave empty — use env
var
    recipient: yourname@gmail.com    # where to send alerts
    min_level: HIGH
    cooldown_seconds: 300
```

Step 4 — Restart and Test

```
sudo systemctl restart netwatchm
journalctl -u netwatchm -f | grep -i email
```

When the next HIGH alert fires, you will receive an email within seconds.

Phase 12 Complete — You now have:

- ✔ A Gmail App Password created and stored securely
- ✔ Email alerts enabled for HIGH and CRITICAL threats
- ✔ A 5-minute cooldown so your inbox isn't flooded

PHASE 13

Running the Tests

13

Verify that every part of NetWatchM is working correctly.

NetWatchM has **57 automated tests** that check every detector, the config loader, the inventory system, and all alert handlers. Running them confirms your installation is correct.

1 Go to the project folder

```
cd /home/$USER/ai-projects/netwatchm
```

2 Run all 57 tests

```
uv run pytest tests/ -v
```

The `-v` flag makes pytest print each test name so you can see exactly what is being checked.

3 What success looks like

```
tests/test_port_scan.py::test_port_scan_fires PASSED
tests/test_port_scan.py::test_port_scan_deduplicates
PASSED
tests/test_brute_force.py::test_brute_force_fires PASSED
...
57 passed in 1.23s
```

4 Run a single test file

```
# Only test the port scan detector
uv run pytest tests/test_port_scan.py -v

# Only test the inventory store
uv run pytest tests/test_inventory_store.py -v
```

What Each Test File Covers

Test File	What It Tests
`test_models.py`	Packet, Alert, DeviceRecord data structures
`test_config.py`	YAML loading, default values, env var password
`test_port_scan.py`	Port scan fires correctly and deduplicates
`test_brute_force.py`	Brute force fires on the right ports
`test_exfiltration.py`	Exfiltration fires at the byte threshold
`test_new_ip.py`	New IP respects baseline period
`test_scorer.py`	Threat scorer aggregates and expires alerts
`test_inventory_store.py`	Device store updates, persists, and loads
`test_exporter.py`	CSV export produces correct output
`test_capture.py`	JSON packet parsing edge cases
`test_alerts.py`	All four alert handlers behave correctly

Phase 13 Complete — You can now:

- ✓ Run all 57 tests in one command
- ✓ Identify which component a failing test covers
- ✓ Run individual test files for focused debugging

PHASE 14

Troubleshooting Common Problems

14

Step-by-step fixes for the most common issues.

Problem 1 — "uv: command not found"

Cause: uv is installed but not on your PATH.

Fix:

```
export PATH="$HOME/.local/bin:$PATH"

# Make it permanent:
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

Problem 2 — "tshark: command not found"

Cause: tshark is not installed.

Fix:

```
sudo apt update && sudo apt install tshark
```

Problem 3 — "Permission denied" on network interface

Cause: tshark needs root access to capture packets.

Fix — Option A (easiest): always run with sudo

```
sudo uv run netwatchm --config /etc/netwatchm/netwatchm.yaml
```

Fix — Option B: give tshark the capability so no sudo is needed

```
sudo setcap cap_net_raw,cap_net_admin+eip $(which tshark)
sudo setcap cap_net_raw,cap_net_admin+eip $(which dumpcap)
```

Problem 4 — Service shows "failed" or "inactive"

Cause: A config error or missing file.

Fix:

```
# See exactly what went wrong
journalctl -u netwatchm -n 30

# Check the config file for YAML syntax errors
python3 -c "import yaml; yaml.safe_load(open('/etc/netwatchm/
netwatchm.yaml'))"

# If the config is fine, try restarting
sudo systemctl restart netwatchm
```

Problem 5 — Browser shows "Could not load inventory.json"

Cause: The `netwatchm-web` service stopped (terminal closed, crash, or not yet installed).

Fix — check and restart the service:

```
systemctl status netwatchm-web

# If stopped, start it:
sudo systemctl start netwatchm-web

# If not installed yet, install it first:
sudo cp /home/$USER/ai-projects/netwatchm/netwatchm-web.service
\
```

```
        /etc/systemd/system/netwatchm-web.service
sudo systemctl daemon-reload
sudo systemctl enable --now netwatchm-web
```

Problem 6 — No packets are being captured

Cause: Wrong interface name in config.

Fix:

```
# Find your interface name
ip addr

# Test that tshark can see traffic on it
sudo tshark -i enp6s0 -c 5

# Update the config
sudo nano /etc/netwatchm/netwatchm.yaml
# Change: interface: auto  →  interface: enp6s0

# Restart
sudo systemctl restart netwatchm
```

Problem 7 — Too many false alerts

Cause: Detection thresholds are too sensitive for your network.

Fix: Raise the thresholds in the config file:

```
thresholds:
  port_scan:
    ports_per_window: 30    # was 15 — doubled to reduce false
positives
    window_seconds: 10
  brute_force:
    attempts_per_window: 20 # was 10
    window_seconds: 30
```

Problem 8 — Sound alert not playing

Cause: No audio device available (common on servers or VMs).

Test:

```
python3 -c "import pygame; pygame.mixer.init(); print('Audio
OK')"
```

If this fails, disable the sound alert in config:

```
sound:
    enabled: false
```

Problem 9 — Email alerts not sending

Check each item in order:

```
# 1. Is the env var set?
sudo cat /etc/netwatchm/env

# 2. Is email enabled in config?
grep "enabled" /etc/netwatchm/netwatchm.yaml

# 3. Check logs for email errors
journalctl -u netwatchm | grep -i "email\|smtp\|mail"

# 4. Test SMTP connection manually
python3 -c "
import smtplib
s = smtplib.SMTP('smtp.gmail.com', 587)
s.starttls()
print('SMTP connection: OK')
s.quit()
"
```

Phase 14 Complete — You can now:

- ✓ Diagnose and fix the 9 most common NetWatchM problems
- ✓ Read logs to understand what went wrong

- ✔ Tune thresholds to reduce false positives

PHASE 15

Log Management & Disk Protection

15

Prevent NetWatchM's logs from saturating your disk — and understand what limits are already in place.

The Three Log Sources

NetWatchM produces output in three places. Each one has a different risk level and a different protection mechanism:

Log Source	Location	Default Cap	Risk
Alert log file	`/var/log/netwatchm/netwatchm.log`	10 MB × 6 files = **60 MB max**	Low — already capped in code
systemd journal	`journalctl -u netwatchm`	**No limit by default**	High — can grow to 10% of your disk
Inventory database	`/var/lib/netwatchm/inventory.json`	Bounded by unique device count	Low — small on most networks

🚨 **The systemd journal has no limit by default**

Without configuration, systemd will let the journal grow to 10% of your filesystem or 4 GB — whichever is smaller. During a sustained attack (thousands of alerts per second) the journal can fill your disk in hours. The fix below caps it hard.

Layer 1 — Alert Log File (Already Protected)

The code in `alerts/logfile.py` uses Python's built-in `RotatingFileHandler`. It automatically:

- [ ] Rotates the log when it reaches **10 MB**
- [ ] Keeps a maximum of **5 backup files** (`netwatchm.log.1` … `netwatchm.log.5`)
- [ ] Hard maximum on disk: **60 MB** regardless of how many alerts fire

You can tune both numbers in `/etc/netwatchm/netwatchm.yaml`:

```
alerts:
  log:
    enabled: true
    path: /var/log/netwatchm/netwatchm.log
    max_bytes: 10485760
# 10 MB per file — raise to 20971520 for 20 MB
    backup_count: 5        # files to keep — raise to 10 for
more history
```

Layer 2 — Cap the systemd Journal (Action Required)

The journal captures everything NetWatchM prints to stderr — including all Python log output. Capping it requires a one-time configuration change.

1. **Create the journald drop-in directory**

```
sudo mkdir -p /etc/systemd/journald.conf.d
```

2 Install the limits config

```
sudo cp /home/$USER/ai-projects/netwatchm/netwatchm-
journald.conf \
        /etc/systemd/journald.conf.d/netwatchm.conf
```

This file sets the following limits:

Setting	Value	What it does
`SystemMaxUse`	**200 MB**	Hard cap on total journal disk usage
`SystemMaxFileSize`	**50 MB**	Each journal file rotates at 50 MB
`SystemKeepFree`	**500 MB**	Always keeps 500 MB free — journal backs off before filling disk
`MaxRetentionSec`	**30 days**	Auto-deletes entries older than 30 days
`RateLimitBurst`	**500 / 10 s**	Drops log messages if a service floods more than 500 per 10 s
`Compress`	**yes**	Compresses journal files on disk (typically 3–5× reduction)

3 Apply the new limits

```
sudo systemctl restart systemd-journald
```

4 Vacuum the journal right now

The new limits apply going forward, but old entries already on disk are not removed automatically until the next rotation. Force a cleanup immediately:

```
# Remove entries until journal is under 200 MB
sudo journalctl --vacuum-size=200M

# Remove entries older than 30 days
sudo journalctl --vacuum-time=30d

# Verify new disk usage
journalctl --disk-usage
```

Layer 3 — logrotate Safety Net

As a belt-and-suspenders backup (in case the Python handler ever fails to rotate), install a `logrotate` config for the alert log:

```
sudo cp /home/$USER/ai-projects/netwatchm/netwatchm-logrotate \
        /etc/logrotate.d/netwatchm
```

This runs daily and keeps 7 compressed backups. The `postrotate` hook sends a `HUP` signal to the service so it reopens the log file cleanly after rotation.

Test that logrotate accepts the config without errors:

```
sudo logrotate --debug /etc/logrotate.d/netwatchm
```

Checking Disk Usage at Any Time

```
# Alert log files
ls -lh /var/log/netwatchm/

# Inventory database
ls -lh /var/lib/netwatchm/inventory.json

# systemd journal for NetWatchM
journalctl -u netwatchm --disk-usage
```

```
# Total journal usage across all services
journalctl --disk-usage
```

What Happens During a Heavy Attack

Without limits, a sustained port scan generating 1 000 alerts/second would produce:

Duration	Alerts	Unprotected journal size	With limits
1 minute	60 000	~60 MB	Rate-limited — capped at 500/10 s
1 hour	3 600 000	~3.6 GB 🚨	Still ≤ 200 MB ✅
1 day	86 400 000	Disk full 🚨	Still ≤ 200 MB ✅

💡 **Rate limiting does not drop real alerts**

The journal rate limit only affects the log text sent to journald. The alert handlers (log file, sound, email) run independently — they are never rate-limited by journald. A HIGH alert will still trigger the beep and email even if journald is dropping its own copy of the message.

Phase 15 Complete — You now have:

- ✓ Alert log file capped at 60 MB with automatic rotation (built-in)
- ✓ systemd journal capped at 200 MB with 30-day retention
- ✓ Rate limiting preventing log flooding during attacks
- ✓ logrotate as a daily safety net for the alert log
- ✓ Commands to check and vacuum disk usage at any time

PHASE 16

Service-Down Email Alerts

Get an email the moment `netwatchm` or `netwatchm-web` stops — with an explanation of why.

NetWatchM can send you an email whenever one of its services goes down unexpectedly. The email includes the failure reason, the last 25 log lines, and exact restart instructions.

How It Works

systemd has a built-in hook called `OnFailure=`. When a service exits with an error, systemd automatically starts the unit named in that directive. NetWatchM uses this to trigger a small Python script that composes and sends an alert email.

```
netwatchm.service (crashes or hits start limit) │ │ OnFailure=netwatchm-notify@%n.service ▼ netwatchm-notify@netwatchm.service (oneshot — runs once) │ │ ExecStart=/usr/local/bin/netwatchm-notify netwatchm ▼ notify-down.py 1. reads /etc/netwatchm/env → Gmail password 2. reads /etc/netwatchm/netwatchm.yaml → SMTP settings, recipient 3. runs: journalctl -u netwatchm -n25 → last log lines 4. runs: systemctl show netwatchm → exit code + result 5. maps exit code → human reason 6. sends email via Gmail SMTP │ ▼ 📧 Your inbox — within seconds of the failure
```

What the Email Contains

Field	Example
Subject	`[NetWatchM] ⚠ Service DOWN: netwatchm on ai-rnd-01`
Service	

Field	Example
	Which service failed (`netwatchm` or `netwatchm-web`)
Host	The machine hostname
Time	Exact timestamp of the failure
Result	systemd result code: `exit-code` , `signal` , `start-limit-hit` , etc.
Exit code	The process exit code (e.g. `1` , `137` , `143`)
Why this is happening	Plain-English explanation mapped from the result and exit code
Last 25 log lines	The actual journal output showing the error or traceback
How to recover	Exact commands to check logs and restart

Failure Reason Reference

systemd Result	Exit Code	Plain-English Reason
`exit-code`	1	Unhandled Python exception — check the log traceback
`exit-code`	2	Configuration error — check `netwatchm.yaml` for syntax issues
`exit-code`	127	Executable not found — uv or .venv path is broken
`signal`	137	Force-killed (SIGKILL) — OOM killer or `systemctl kill`
`signal`	143	Graceful shutdown (SIGTERM) — `systemctl stop` or reboot
	—	

systemd Result	Exit Code	Plain-English Reason
`start-limit-hit`		Crashed and restarted too many times — systemd stopped retrying
`watchdog`	—	Process frozen or stuck in an infinite loop
`oom-kill`	—	System ran out of RAM — consider adding swap

First-Time Setup

¡ **Prerequisite: email alerts must already be configured**

The notification script uses the same Gmail App Password and SMTP settings from Phase 12. Complete Phase 12 before following the steps below.

1 Install the notification script

```
sudo cp /home/$USER/ai-projects/netwatchm/scripts/notify-down.py \
        /usr/local/bin/netwatchm-notify
sudo chmod +x /usr/local/bin/netwatchm-notify
```

2 Install the systemd template unit

```
sudo cp /home/$USER/ai-projects/netwatchm/netwatchm-notify@.service \
        /etc/systemd/system/netwatchm-notify@.service
sudo systemctl daemon-reload
```

3 Re-install the monitor service to add the OnFailure hook

```
cd /home/$USER/ai-projects/netwatchm
sudo env NETWATCHM_EMAIL_PASSWORD="$(sudo grep
NETWATCHM_EMAIL_PASSWORD /etc/netwatchm/env | cut -d= -
f2)" \
    .venv/bin/netwatchm --config /etc/netwatchm/
netwatchm.yaml --install-service
```

This rewrites `/etc/systemd/system/netwatchm.service` with the `OnFailure=netwatchm-notify@%n.service` line now included.

4 Update the web dashboard service

```
sudo cp /home/$USER/ai-projects/netwatchm/netwatchm-
web.service \
        /etc/systemd/system/netwatchm-web.service
sudo systemctl daemon-reload
sudo systemctl restart netwatchm-web
```

5 Test the alert works

```
# Run the script manually against a real service name
sudo /usr/local/bin/netwatchm-notify netwatchm
```

Check your inbox — you should receive a test email within a few seconds. The email will show the current service state (which is running, not crashed, since this is just a manual test).

💡 **The installer handles all of this automatically**

If you run `sudo bash install.sh` on a fresh machine, step 10 of the installer copies the script, installs the template service, and registers `OnFailure=` in both service units automatically. No manual steps needed.

Phase 16 Complete — You now have:

- ✓ Email alert fired automatically when `netwatchm` stops

- Email alert fired automatically when `netwatchm-web` stops
- Plain-English failure reason mapped from exit code and systemd result
- Last 25 log lines included so you can diagnose without logging in
- Exact restart commands included in every alert email

PHASE 17

PowerShell Log Watcher

A read-only PowerShell function that streams journalctl output and raises a high-severity console alert the instant the word `HIGH` appears.

Design Constraints

This watcher was built under a strict secure-coding rule:

Rule	How it is enforced
Non-destructive	No `systemctl`, `kill`, `rm`, or file writes inside the function
Read-only	`journalctl --follow` only reads the kernel ring buffer. No `--rotate`, `--vacuum`, or `--flush` flags are used
No system commands	Only `journalctl` (log reader) is invoked. No bash, sudo, or state-altering calls
Alert only	Output goes to `Write-Host` (console) or a pipeable `PSCustomObject` — the caller decides what to do with it
Case-sensitive match	`[regex]::IsMatch($line, '\bHIGH\b')` — only exact uppercase `HIGH` triggers an alert

How It Works

```
journalctl -u netwatchm --follow --no-pager --output short-iso |
(read-only stdout stream — one line per log entry) | ▼ ForEach-
Object { $line = $_ } | ├── [regex]::IsMatch($line, '\bHIGH\b') ? |
| | ├── NO → continue silently | | | └── YES → | Write-Host
coloured alert box (console only) | PSCustomObject { Timestamp,
```

```
Severity, Unit, LogLine } | | | └── emitted to success stream if -
EmitObjects is set | caller pipes to CSV, email, etc. ▼ Ctrl+C →
clean exit, no cleanup needed
```

The Function

```
function Watch-NetWatchMLogs {
    <#
    .SYNOPSIS
        Watches journalctl output and raises a HIGH-severity
alert
        when the exact uppercase keyword "HIGH" appears in a
log line.

    .PARAMETER Unit
        The systemd unit to follow. Defaults to 'netwatchm'.

    .PARAMETER EmitObjects
        When set, each HIGH alert is also emitted as a
PSCustomObject
        on the success stream so callers can pipe it further.
    #>
    [CmdletBinding()]
    param(
        [Parameter()]
        [ValidateNotNullOrEmpty()]
        [string] $Unit = 'netwatchm',

        [Parameter()]
        [switch] $EmitObjects
    )

    # Validate journalctl is available (read-only check)
    if (-not (Get-Command journalctl -ErrorAction
SilentlyContinue)) {
        Write-Error "journalctl not found. Requires a systemd-
based Linux host."
        return
    }

    # Read-only journalctl arguments — no state-altering flags
    [string[]] $jArgs = @(
        '--unit',   $Unit,
        '--follow',
        '--no-pager',
        '--output', 'short-iso'
    )
```

```
    Write-Host "Watching journalctl -u $Unit  (Ctrl+C to stop)"
-ForegroundColor Cyan

    try {
        & journalctl @jArgs | ForEach-Object {
            [string] $line = $_

            # Case-sensitive exact-word match for uppercase
"HIGH"
            if ([regex]::IsMatch($line, '\bHIGH\b')) {

                [datetime] $detected  = [datetime]::Now
                [string]   $timestamp =
$detected.ToString('yyyy-MM-dd HH:mm:ss')

                # Console alert — non-destructive output only
                Write-Host ""
                Write-Host
"╔══════════════════════════════════════════════════════╗" `
                    -ForegroundColor Red
                Write-Host "║  ⚠  HIGH-SEVERITY ALERT
DETECTED                ║" `
                    -ForegroundColor Red
                Write-Host
"╠══════════════════════════════════════════════════════╣" `
                    -ForegroundColor Red
                Write-Host "║  Time   : $timestamp"  -
ForegroundColor Yellow
                Write-Host "║  Unit   : $Unit"       -
ForegroundColor Yellow
                Write-Host "║  Source : journalctl (read-only
stream)" `
                    -ForegroundColor Yellow
                Write-Host
"╠══════════════════════════════════════════════════════╣" `
                    -ForegroundColor Red
                Write-Host "║  Log line:"            -
ForegroundColor White
                Write-Host "║  $line"                -
ForegroundColor White
                Write-Host
"╚══════════════════════════════════════════════════════╝" `
                    -ForegroundColor Red
                Write-Host ""

                # Structured object to success stream
(pipeable)
                if ($EmitObjects) {
                    [PSCustomObject] @{
```

```
                        Timestamp = $detected
                        Severity  = 'HIGH'
                        Unit      = $Unit
                        LogLine   = $line
                    }
                }
            }
        }
    }
    catch [System.OperationCanceledException] {
        Write-Host "`nWatch stopped by user." -ForegroundColor
DarkGray
    }
    catch {
        Write-Error "Stream error: $_"
    }
}
```

Saving the Function

1 Install PowerShell Core on Linux

```
# Ubuntu / Debian
sudo apt install -y powershell

# Verify
pwsh --version
```

2 Save the function to your PowerShell profile

```
# Open (or create) your PowerShell profile
pwsh -Command "New-Item -Force $PROFILE | Out-Null; code
$PROFILE"

# Or save to a standalone script file
cp /home/$USER/ai-projects/netwatchm/scripts/Watch-
NetWatchMLogs.ps1 ~/
```

3 Load and run

```
pwsh

# Dot-source to load the function into the session
. ~/Watch-NetWatchMLogs.ps1

# Basic usage — alerts print to console
Watch-NetWatchMLogs

# Specify a different unit
Watch-NetWatchMLogs -Unit netwatchm-web

# Emit structured objects and save to CSV
Watch-NetWatchMLogs -EmitObjects |
    Export-Csv ~/high-alerts.csv -Append -
NoTypeInformation
```

Usage Examples

Command	What it does
`Watch-NetWatchMLogs`	Watch `netwatchm`, print coloured alert box on HIGH
`Watch-NetWatchMLogs -Unit netwatchm-web`	Watch the web dashboard service instead
`Watch-NetWatchMLogs -EmitObjects`	Emit `PSCustomObject` per alert (pipeable)
`... | Export-Csv alerts.csv -Append`	Save all HIGH alerts to a CSV log
`... | ConvertTo-Json | Out-File alerts.jsonl -Append`	Save as JSON Lines for log ingestion tools
`Ctrl+C`	Stop the watcher cleanly

What a HIGH alert looks like in the console

When `HIGH` is detected in a log line, the console prints:

```
╔══════════════════════════════════════════════════════╗
║  ⚠  HIGH-SEVERITY ALERT DETECTED                     ║
╠══════════════════════════════════════════════════════╣
║  Time   : 2026-02-21 19:45:03
║  Unit   : netwatchm
║  Source : journalctl (read-only stream)
╠══════════════════════════════════════════════════════╣
║  Log line:
║  2026-02-21T19:45:03 ai-rnd-01 netwatchm: HIGH
PORT_SCAN ...
╚══════════════════════════════════════════════════════╝
```

Phase 17 Complete — You now have:

- ✓ A read-only, non-destructive PowerShell watcher for NetWatchM logs
- ✓ Instant coloured console alert when `HIGH` appears in a log line
- ✓ Case-sensitive exact-word match — no false positives from "higher" or "highlight"
- ✓ Pipeable `PSCustomObject` output for CSV, JSON, or email forwarding
- ✓ Clean Ctrl+C exit with no system state changes

APPENDIX

Glossary & Quick Reference

Key terms and the commands you will use most often.

Glossary

Term	Plain-English Definition
asyncio	Python's way of doing many things at once without using multiple threads. NetWatchM uses it so packet capture, detection, alerting, and the UI all run simultaneously.
tshark	The command-line version of Wireshark. It reads raw network packets and outputs them as JSON lines that NetWatchM parses.
NIC	Network Interface Card — the hardware (physical or virtual) that connects your computer to a network. Examples: `enp6s0`, `eth0`, `wlan0`.
promiscuous mode	A mode where the NIC captures ALL packets on the network segment, not just packets addressed to your machine. Required for network monitoring.
port	A numbered "door" on a computer that a specific service listens on. Port 22 = SSH, port 80 = HTTP, port 443 = HTTPS.
port scan	Rapidly trying to connect to many ports to see which ones are open. A common first step before an attack.
brute force	Trying thousands of passwords automatically until one works.
exfiltration	

Term	Plain-English Definition
	Sneaking data out of a network — uploading stolen files, screenshots, or credentials to an attacker's server.
sliding window	A time-based counter that tracks events within the last N seconds. Old events "slide out" and are forgotten. NetWatchM's detectors use this to count ports or bytes in a time window.
ThreatLevel	LOW → MEDIUM → HIGH → CRITICAL. Each detector assigns a level; the ThreatScorer shows the highest active one.
inventory.json	The on-disk database of every device NetWatchM has seen, updated every 60 seconds.
systemd	Linux's service manager. It starts NetWatchM on boot, restarts it if it crashes, and collects its logs.
uv	A fast Python package manager from Astral. Replaces pip + venv. NetWatchM uses it to install dependencies and run the app.
App Password	A special 16-character password generated by Google that lets third-party apps send email via Gmail without using your real password.
Rich	A Python library that creates colourful tables and live dashboards in the terminal.
NDJSON	Newline-Delimited JSON — one complete JSON object per line. tshark outputs packets in this format with `-T ek` .

Quick Reference — Most Used Commands

```
── SERVICE MANAGEMENT ──────────────────────────────────────

sudo systemctl start netwatchm          # start the service
sudo systemctl stop netwatchm           # stop the service
sudo systemctl restart netwatchm        # restart (after config
```

```
changes)
systemctl status netwatchm             # is it running?
journalctl -u netwatchm -f             # follow live logs

── RUN INTERACTIVELY ──────────────────────────────────────
sudo uv run netwatchm --config /etc/netwatchm/netwatchm.yaml
sudo uv run netwatchm --config /etc/netwatchm/netwatchm.yaml --
no-ui

── DEVICE INVENTORY ───────────────────────────────────────
uv run netwatchm inventory
uv run netwatchm inventory --filter 192.168
uv run netwatchm inventory --sort-by threat
uv run netwatchm inventory --export /tmp/devices.csv

── WEB DASHBOARD (netwatchm-web service) ──────────────────
# First-time install
sudo cp report.html /var/lib/netwatchm/
sudo cp netwatchm-web.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now netwatchm-web
# Open: http://localhost:8765/report.html

# Service management
systemctl status netwatchm-web
sudo systemctl restart netwatchm-web
journalctl -u netwatchm-web -f

── TESTS ──────────────────────────────────────────────────
uv run pytest tests/ -v                # run all 57 tests
uv run pytest tests/test_port_scan.py  # run one file

── LOGS ───────────────────────────────────────────────────
tail -f /var/log/netwatchm/netwatchm.log
cat /var/lib/netwatchm/inventory.json | python3 -m json.tool

── POWERSHELL LOG WATCHER (Phase 17) ──────────────────────
# Start PowerShell Core
pwsh

# Load the function and run
. ~/Watch-NetWatchMLogs.ps1
Watch-NetWatchMLogs                          # console alerts
Watch-NetWatchMLogs -Unit netwatchm-web      # watch web
```

```
service
Watch-NetWatchMLogs -EmitObjects | Export-Csv ~/alerts.csv -
Append

── SERVICE-DOWN ALERTS (Phase 16)
──────────────────────────────────

# Install
sudo cp scripts/notify-down.py /usr/local/bin/netwatchm-notify
sudo chmod +x /usr/local/bin/netwatchm-notify
sudo cp netwatchm-notify@.service /etc/systemd/system/
sudo systemctl daemon-reload

# Test manually
sudo /usr/local/bin/netwatchm-notify netwatchm

── LOG MANAGEMENT (Phase 15)
─────────────────────────────────────

# Check disk usage
ls -lh /var/log/netwatchm/
journalctl -u netwatchm --disk-usage
journalctl --disk-usage

# Apply journal limits (one-time setup)
sudo mkdir -p /etc/systemd/journald.conf.d
sudo cp netwatchm-journald.conf /etc/systemd/journald.conf.d/
netwatchm.conf
sudo systemctl restart systemd-journald

# Vacuum journal immediately
sudo journalctl --vacuum-size=200M
sudo journalctl --vacuum-time=30d

# Install logrotate safety net
sudo cp netwatchm-logrotate /etc/logrotate.d/netwatchm
sudo logrotate --debug /etc/logrotate.d/netwatchm
```